

Time Complexity Solutions

A comprehensive reference guide with step-by-step solutions for Iterative, Recursive, and Recurrence Relation analysis.

1. Iterative Loops (Q1 - Q13)

Q1

```
for(int i = 0; i < n; i++) {  
    cout << i;  
}
```

Explanation: The loop runs from 0 to $n-1$ and increments by 1 in each iteration. It will execute a total of n times.

Time Complexity: $O(n)$

Q2

```
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < n; j++) {  
        cout << "*";  
    }  
}
```

Explanation: The outer loop runs n times. For every single iteration of the outer loop, the inner loop also runs n times. Total iterations = $n \times n = n^2$.

Time Complexity: $O(n^2)$

Q3

```
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < i; j++) {  
        cout << "*";  
    }  
}
```

Explanation: The inner loop depends on i .

- When $i = 0$, the inner loop runs 0 times.
- When $i = 1$, the inner loop runs 1 time.

Total sum = $0 + 1 + 2 + \dots + (n-1) = n(n-1)/2 = (n^2 - n)/2$.

Time Complexity: $O(n^2)$

Q4 & Q5

```
// Q4: for(int i = 1; i < n; i *= 2)  
// Q5: for(int i = n; i > 1; i /= 2)
```

Explanation: In both cases, i is either being multiplied or divided by 2. When a variable changes at an exponential rate, the steps are logarithmic ($2^k = n \rightarrow k = \log_2 n$).

Time Complexity: $O(\log n)$

Q6

```
for(int i = 1; i <= n; i++) {  
    for(int j = 1; j <= log(n); j++) {  
        cout << "*";  
    }  
}
```

Explanation: The outer loop runs n times. The inner loop runs a fixed $\log n$ times. Total operations = $n \times \log n$.

Time Complexity: $O(n \log n)$

Q7

```
while (n > 0) {  
    n--;  
}
```

Explanation: The loop starts from n , goes down to 0 , and decrements by 1 at each step. It will run exactly n times.

Time Complexity: $O(n)$

Q8

```
while (n > 1) {  
    n /= 2;  
}
```

Explanation: In each step, n is halved ($n/2$) until it becomes 1 . This is a standard binary division pattern.

Time Complexity: $O(\log n)$

Q9

<

```
for(int i = 1; i <= n; i++) {}  
for(int j = 1; j <= n; j++) {}
```

Explanation: Both loops are independent and run one after the other (sequentially). Total time = $n + n = 2n$. Constants are neglected in Big-O notation.

Time Complexity: $O(n)$

Q10

<

```
for(int i = 1; i <= n; i++) {}  
for(int j = 1; j <= n * n; j++) {}
```

Explanation: The first loop runs n times and the second independent loop runs n^2 times. The highest power dominates.

Time Complexity: $O(n^2)$

Q11

<

```
for(int i = 1; i <= n; i++) {  
    for(int j = 1; j <= n * n; j++) {}  
}
```

Explanation: This is a nested loop. The outer loop runs n times, and each time the inner loop runs n^2 times. Total iterations = $n \times n^2 = n^3$.

Time Complexity: $O(n^3)$

Q12

<

```
for(int i = 1; i <= n; i *= 3) {}
```

Explanation: i is multiplied by 3 each time ($3^k = n \rightarrow k = \log_3 n$).

Time Complexity: $O(\log n)$

Q13

<

```
for(int i = 1; i <= n; i++) {  
    for(int j = 1; j <= i * i; j++) {}  
}
```

Explanation: The inner loop runs up to i^2 . Total sum = $1^2 + 2^2 + \dots + n^2 = n(n+1)(2n+1)/6 \approx n^3/3$.

Time Complexity: $O(n^3)$

2. Recursive Functions (Q14 - Q20)

Q14

```
void fun(int n) {  
    if(n == 0) return;  
    fun(n - 1);  
}
```

Explanation: This is linear recursion. For each call, the function calls itself with $n-1$ until it reaches zero.

Time Complexity: $O(n)$

Q15

```
void fun(int n) {
    if(n <= 1) return;
    fun(n / 2);
}
```

Explanation: The input size is halved in each recursive call ($n \rightarrow n/2 \rightarrow n/4 \dots$).

Time Complexity: $O(\log n)$

Q16 & Q17 (Fibonacci Type)

```
// fun(n-1) + fun(n-1)
// return fib(n-1) + fib(n-2)
```

Explanation: Each single call generates 2 new calls, creating a binary tree structure with a depth of n . Total nodes $\approx 2^n$.

Time Complexity: $O(2^n)$

Q18

```
int fact(int n) {
    if(n <= 1) return 1;
    return n * fact(n - 1);
}
```

Explanation: The factorial function forms a linear chain that makes recursive calls starting from n down to 1.

Time Complexity: $O(n)$

Q19 & Q20

Explanation:

- **Q19:** There is a single printing operation after $fun(n-1) \rightarrow O(n)$.
- **Q20:** There is a single printing operation after $fun(n/2) \rightarrow O(\log n)$.

Time Complexity: Q19: $O(n)$, Q20: $O(\log n)$

3. Recurrence Relations (Q21 - Q25)

Question	Recurrence Relation	Solving Method / Solution	Time Complexity
Q21	$T(n) = T(n-1) + 1$	Substitution: $1 + 1 + \dots + 1$ (n times)	$O(n)$
Q22	$T(n) = T(n-1) + n$	Sum of first n natural numbers: $n(n+1)/2$	$O(n^2)$
Q23	$T(n) = T(n/2) + 1$	Master Theorem Case 2 (Binary Search pattern)	$O(\log n)$
Q24	$T(n) = 2T(n-1) + 1$	Substitution / Tree method: $1 + 2 + 4 + \dots + 2^{n-1}$	$O(2^n)$
Q25	$T(n) = T(n/2) + n$	Master Theorem Case 3 (Decreasing geometric series)	$O(n)$

4. Bonus GATE-Level Questions (Q26 - Q30)

Q26

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= i; j *= 2) {}  
}
```

Explanation: The outer loop goes from 1 to n . The inner loop runs $\log i$ times. Total sum = $\log 1 + \log 2 + \dots + \log n = \log(n!) \approx n \log n$.

Time Complexity: $O(n \log n)$

Q27

```
for (int i = 1; i <= n; i *= 2) {  
    for (int j = 1; j <= n; j++) {}  
}
```

Explanation: The outer loop runs $\log n$ times (because of $i *= 2$). The inner loop is independent and will run a full n times every instance.

Time Complexity: $O(n \log n)$

Q28

```
for (int i = n; i >= 1; i--) {  
    for (int j = i; j >= 1; j--) {}  
}
```

Explanation: The inner loop runs n times first, then $n-1$, then $n-2$, and so on. Sum $\approx n^2/2$.

Time Complexity: $O(n^2)$

Q29

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= n; j *= 2) {}  
}
```

Explanation: The outer loop runs n times. The inner loop runs up to $\log n$ times each instance. Total = $n \times \log n$.

Time Complexity: $O(n \log n)$

Q30

```
void fun(int n) {  
    if(n <= 1) return;  
    fun(n / 2);  
    fun(n / 2);  
}
```

Explanation: Its recurrence relation will be: $T(n) = 2T(n/2) + 1$. Applying the Master Theorem, $n^{\log_2 2} = n^1 = n$. Since $n > 1$ (the work done per split is dominated by the tree leaves), the answer is $O(n)$.

Time Complexity: $O(n)$